

FastFeast Data Pipeline

End-to-End Workflow & Architecture

From raw files on disk → validated data → PostgreSQL star-schema warehouse → SLA analytics

1. Big Picture

FastFeast is a food delivery platform. Every day it exports operational data from its OLTP systems into flat files (CSV and JSON). Our pipeline picks up those files, cleans them, validates them, and loads them into a PostgreSQL data warehouse (OLAP) where SLA breach metrics and business analytics are computed.

The pipeline runs continuously. Batch dimension data arrives once per day. Transaction data (orders, tickets, events) arrives in micro-batches throughout the day at irregular intervals.



2. Data Sources — What Comes In

2.1 Batch Files (once per day)

FastFeast exports a full snapshot of all dimension/master data every morning to:

```
data\input\batch\YYYY-MM-DD\
```

File	Format	Contains
customers.csv	CSV	500+ customers — name, email, phone, region, segment
drivers.csv	CSV	100+ drivers — name, phone, national ID, vehicle, metrics
agents.csv	CSV	30 support agents — name, email, team, skill level, KPIs
restaurants.json	JSON	58 restaurants — name, category, region, rating, prep time
cities.json	JSON	3 cities — Cairo, Giza, Alexandria
regions.csv	CSV	18 delivery regions with base fees
segments.csv	CSV	2 customer segments — Regular, VIP
categories.csv	CSV	10 restaurant categories
teams.csv	CSV	5 support teams
reasons.csv	CSV	10 complaint reasons with severity and refund %

channels.csv	CSV	4 support channels — app, chat, phone, email
priorities.csv	CSV	4 ticket priorities P1–P4 with SLA thresholds
reason_categories.csv	CSV	3 reason categories — Delivery, Food, Payment

2.2 Stream Files (micro-batch, throughout the day)

Transaction data is exported at irregular intervals to hourly subdirectories:

```
data\input\stream\YYYY-MM-DD\HH\
```

File	Format	Contains
orders.json	JSON	50–360 orders/hour — customer, restaurant, driver, amounts, status, timestamps
tickets.csv	CSV	~12% of orders generate a support ticket — agent, reason, priority, SLA due times, refund
ticket_events.json	JSON	3–5 lifecycle events per ticket — Open → InProgress → Resolved/Closed (8% get Reopened)

⚠ Stream files contain intentional data quality issues: duplicates (0.5–3%), null values (1–5%), invalid dates (0.5–2%), negative amounts (0.2–1%), and orphan FK references (1–3%). The pipeline handles all of these.

3. Pipeline Execution Flow — Step by Step

Phase 0 — Startup (main.py → orchestrator.py)

1. Read pipeline_config.yaml + .env — load all settings into memory
2. Test PostgreSQL connection — retry up to 3 times with 2-second delays
3. Run CREATE TABLE IF NOT EXISTS for all 17 warehouse tables — safe to run on every startup
4. Hand control to BatchIngestor

Phase 1 — Batch Ingestion (batch_ingestion.py)

Processes all 13 files in the batch directory in a strict order — lookup tables first (they have no FK dependencies), entity tables second (they reference lookups).

#	Action	Detail
1	Idempotency check	file_tracker.py checks SQLite — if this file was already processed successfully with the same MD5 hash, skip it entirely
2	Read file	file_reader.py reads CSV or JSON into a pandas DataFrame — handles encoding issues, empty files, malformed JSON without crashing
3	Schema validate	schema_validator.py checks all required columns are present. If any required column is missing → skip the entire file, log error, send async alert
4	Data validate	data_validator.py applies 6 checks in order: deduplicate on PK → null enforcement → email regex → phone regex → numeric range checks → required date fields. Bad rows are dropped and sent to quarantine

5	PII masking	pii_handler.py replaces email, phone, and national_id with SHA-256 hashes before any data touches the warehouse
6	Upsert to warehouse	loader.py runs INSERT ... ON CONFLICT DO UPDATE — identical rows on re-run produce the same result
7	Quality metrics	persist_metrics() writes one row to quality_log — total, valid, rejected, orphan counts, latency
8	Mark done	file_tracker.mark_done() records the file path + MD5 hash — future runs skip this file

After all 13 batch files are processed, the orchestrator loads all PKs from the warehouse dim tables into memory (customer_ids, driver_ids, restaurant_ids, etc.) — these are used for FK validation in the stream phase.

Phase 2 — Stream Ingestion (stream_ingestion.py)

A background daemon thread starts and polls the stream directory every 30 seconds (configurable). Within each hourly slot it always processes files in this exact order:

- orders.json → then refresh order ID cache
- tickets.csv → then refresh ticket ID cache
- ticket_events.json

This order is mandatory — tickets reference orders, and events reference tickets. Processing out of order would cause false orphan rejections.

For each stream file, the same 8-step pipeline runs as batch PLUS one extra step between steps 4 and 5:

#	Action	Detail
4b	Referential validate	referential_validator.py checks every FK column against the in-memory ID sets loaded from the warehouse. Orphan rows (e.g. orders referencing a customer_id that doesn't exist in dim_customer) are split out, quarantined, and counted separately. If orphan rate > 5%, an async alert is fired.

Phase 3 — SLA Calculation (sla_calculator.py)

After every batch of tickets is loaded, a single SQL UPDATE runs against fact_ticket:

```
UPDATE fact_ticket SET
  is_first_response_breached = (first_response_at > sla_first_due_at),
  is_resolution_breached     = (resolved_at > sla_resolve_due_at),
  first_response_seconds     = EXTRACT(EPOCH FROM first_response_at - created_at),
  resolution_minutes         = EXTRACT(EPOCH FROM resolved_at - created_at) / 60
WHERE is_first_response_breached IS NULL;
```

The WHERE clause ensures only newly inserted rows are processed — existing rows are never recalculated. SLA thresholds come from the priorities table: first response = 1 minute, resolution = 15 minutes.

Six analytics views are then refreshed automatically:

- v_sla_daily — breach rates, avg response times, refunds by day
- v_sla_by_restaurant — which restaurants generate the most SLA breaches
- v_sla_by_driver — which drivers cause the most complaints

- v_sla_by_region — breach rates by city and region
- v_revenue_impact — refund totals and complaint rate per 1,000 orders
- v_reopen_rate — % of tickets that get reopened after resolution

4. Data Quality — What the Pipeline Fixes

Issue	Example in source data	Pipeline action	Where logged
Duplicate rows	Same order_id in two files	Keep first, drop rest	quality_log.duplicate_count
Null in required field	customer_id is null	Drop row → quarantine	quality_log.null_violation_count
Invalid email	'notanemail', '@nodomain.com'	Drop row → quarantine	quality_log.invalid_format_count
Invalid phone	'abc12345', '020xxxxxxx'	Drop row → quarantine	quality_log.invalid_format_count
Numeric out of range	rating = -1 or 6, on_time_rate = 1.5	Drop row → quarantine	quality_log.invalid_format_count
Invalid date	'N/A', 'invalid', empty string	Drop row → quarantine	quality_log.invalid_format_count
Missing required column	File has no order_id column	Skip entire file → alert	logs\pipeline.log
FK orphan reference	order references customer_id=99999 (not in warehouse)	Quarantine orphan rows → track rate → alert if >5%	quality_log.orphan_count
Raw PII in source	email = 'ahmed@gmail.com'	Hash to SHA-256 before write	Never stored raw

5. Warehouse Schema — Star Schema

13 dimension tables, 3 fact tables, 1 audit table. All tables use INSERT ... ON CONFLICT DO UPDATE (upsert) so re-running the pipeline never creates duplicates.

Dimension Tables (loaded from batch)

Table	Key Columns	Source
dim_city	city_id (PK), city_name, country, timezone	cities.json
dim_region	region_id (PK), region_name, city_id, delivery_base_fee	regions.csv
dim_segment	segment_id (PK), segment_name, discount_pct, priority_support	segments.csv
dim_category	category_id (PK), category_name	categories.csv
dim_team	team_id (PK), team_name	teams.csv
dim_reason_category	reason_category_id (PK), category_name	reason_categories.csv

dim_reason	reason_id (PK), reason_name, reason_category_id, severity_level, typical_refund_pct	reasons.csv
dim_channel	channel_id (PK), channel_name	channels.csv
dim_priority	priority_id (PK), priority_code, priority_name, sla_first_response_min, sla_resolution_min	priorities.csv
dim_customer	customer_id (PK), full_name, email_hash, phone_hash, region_id, segment_id	customers.csv
dim_restaurant	restaurant_id (PK), restaurant_name, region_id, category_id, price_tier, rating_avg	restaurants.json
dim_driver	driver_id (PK), driver_name, phone_hash, national_id_hash, region_id, vehicle_type, on_time_rate	drivers.csv
dim_agent	agent_id (PK), agent_name, email_hash, phone_hash, team_id, skill_level, resolution_rate	agents.csv

Fact Tables (loaded from stream)

Table	Key Columns	Source
fact_order	order_id (PK, UUID), customer_id, restaurant_id, driver_id, region_id, order_amount, delivery_fee, total_amount, order_status, payment_method, order_created_at, delivered_at	orders.json
fact_ticket	ticket_id (PK, UUID), order_id, agent_id, reason_id, priority_id, channel_id, status, refund_amount, created_at, sla_first_due_at, sla_resolve_due_at, is_first_response_breached*, is_resolution_breached*, first_response_seconds*, resolution_minutes*	tickets.csv (* = computed by SLA calculator after load)
fact_ticket_event	event_id (PK, UUID), ticket_id, agent_id, event_ts, old_status, new_status, notes	ticket_events.json

Audit Table

Table	Key Columns
quality_log	run_id (PK), run_date, run_timestamp, table_name, source_file, total_records, valid_records, rejected_records, duplicate_count, null_violation_count, invalid_format_count, orphan_count, orphan_rate, processing_latency_ms, status

6. Orphan References — How They Are Created and Handled

Orphan references are one of the most realistic data engineering challenges in this pipeline. Here is exactly how they are created:

- 5. generate_batch_data.py runs in the morning — takes a snapshot of customers and drivers at that moment
- 6. add_new_customers.py and add_new_drivers.py run later in the day — they add new people to the master data but NOT to the batch snapshot
- 7. generate_stream_data.py generates orders that may reference these new customer/driver IDs
- 8. The pipeline sees these IDs in the orders but cannot find them in dim_customer or dim_driver — these are orphans

The pipeline handles orphans by:

- Splitting the DataFrame into valid rows and orphan rows
- Loading valid rows into fact_order
- Writing orphan rows to data\quarantine\YYYY-MM-DD\orders_orphan_reference_HH-MM-SS.csv
- Recording orphan_count and orphan_rate in quality_log
- Firing an async email alert if orphan_rate > 5% (configurable threshold)

7. Fault Tolerance — The Pipeline Never Crashes

Every error is caught, logged, and skipped. The pipeline continues to the next file or row. Here is the exact behaviour at each failure point:

Failure	Pipeline action	Alert sent?	File marked?
DB unreachable at startup	Log error + exit (only hard stop)	Yes	N/A
File not found	Log warning + skip	No	No
File parse error (bad JSON/CSV)	Log error + skip file	Yes	failed
Missing required column	Log error + skip file	Yes	failed
Row fails null/email/phone/range check	Drop row + quarantine + continue	No	success (partial)
FK orphan row	Quarantine + count + continue valid rows	If rate > 5%	success (partial)
DB write (upsert) fails	Log error + skip file	Yes	failed (will retry next run)
Alert email fails to send	Log the alert failure + continue	N/A	unaffected
Quality metrics write fails	Log warning + continue	No	unaffected

8. Idempotency — Safe to Re-Run

Running the pipeline twice for the same date produces identical results in the warehouse. This is guaranteed at two levels:

- **File level** — SQLite `file_tracker.db` records every processed file by path and MD5 hash. On re-run, if the file hasn't changed, it is skipped entirely.
- **Row level** — All warehouse inserts use `INSERT ... ON CONFLICT DO UPDATE`. Inserting a row that already exists updates it with the same values — no duplicates ever appear.

If a file was previously marked as 'failed' in the tracker, it will be retried on the next run automatically. To force a full reprocess of all files, simply delete `file_tracker.db`.

9. Module Map — What Each File Does

File	Responsibility
<code>config/settings.py</code>	Loads <code>pipeline_config.yaml</code> , expands <code>\${ENV_VAR}</code> from <code>.env</code> , exposes <code>settings.database.host</code> etc.
<code>config/pipeline_config.yaml</code>	Single source of truth for all tunable values — DB, paths, SLA, alert thresholds
<code>src/utis/logger.py</code>	Rotating JSON logger — every line is a valid JSON object written to <code>logs/pipeline.log</code>
<code>src/utis/alerter.py</code>	<code>send_alert_async()</code> — fires failure email in a background daemon thread, never blocks the pipeline
<code>src/utis/quarantine.py</code>	<code>quarantine_rows()</code> — writes rejected rows to <code>data/quarantine/YYYY-MM-DD/</code> with rejection reason and timestamp
<code>src/utis/date_utils.py</code>	<code>parse_datetime()</code> — handles all sentinel garbage values (N/A, invalid, empty) safely
<code>src/ingestion/file_reader.py</code>	<code>read_csv()</code> / <code>read_json()</code> — always return a DataFrame, never raise an exception
<code>src/ingestion/file_tracker.py</code>	FileTracker — SQLite-backed, tracks path + MD5. <code>is_processed()</code> / <code>mark_done()</code>
<code>src/ingestion/batch_ingestion.py</code>	<code>BatchIngestor.run(date)</code> — 8-step pipeline for all 13 batch files in correct order
<code>src/ingestion/stream_ingestion.py</code>	StreamWatcher — daemon thread polling stream dirs, enforces <code>orders→tickets→events</code> order
<code>src/validation/schema_validator.py</code>	SCHEMA_REGISTRY for all 16 tables. File-level check — missing columns = skip file
<code>src/validation/data_validator.py</code>	Row-level: dedup → nulls → email regex → phone regex → numeric ranges → date validity
<code>src/validation/pii_handler.py</code>	<code>mask_pii()</code> — SHA-256 hash email/phone/national_id, drop original columns
<code>src/validation/referential_validator.py</code>	ReferentialValidator — loads ID sets from warehouse, <code>check_orders/tickets/events()</code>
<code>src/warehouse/connection.py</code>	<code>get_connection()</code> — context manager with 3x retry, auto-commit/rollback
<code>src/warehouse/schema_ddl.py</code>	<code>create_all_tables()</code> — 17 CREATE TABLE IF NOT EXISTS + 16 indexes
<code>src/warehouse/loader.py</code>	<code>upsert()</code> — <code>INSERT ... ON CONFLICT DO UPDATE</code> for all tables via <code>execute_batch</code>

src/analytics/sla_calculator.py	refresh_sla() — UPDATE breach flags + CREATE OR REPLACE VIEW for 6 analytics views
src/analytics/quality_metrics.py	persist_metrics() — write quality_log row after each file
pipeline/orchestrator.py	Pipeline.run() — wires all stages: DB check → DDL → batch → load IDs → stream thread
main.py	Entry point — 4 modes: --check, --batch-only, --date (full run), --report-only

10. Analytics — What You Can Query

After the pipeline runs, these views are ready to query in PostgreSQL:

View	Answers
v_sla_daily	By day: total tickets, first-response breach %, resolution breach %, avg response time, avg resolution time, total refunds
v_sla_by_restaurant	By restaurant: breach rate, avg resolution time, total refunds — sorted by worst breach rate
v_sla_by_driver	By driver: tickets caused, breach rate, total refunds, vehicle type, on-time rate
v_sla_by_region	By city and region: first-response breach %, resolution breach %, total refunds
v_revenue_impact	By day: total revenue, total refunds, refund % of revenue, complaint rate per 1,000 orders
v_reopen_rate	By day: total tickets, how many were reopened, reopen rate %
quality_log	Per file: all quality counters — use to audit pipeline health for any date

All SLA breach calculations happen entirely in PostgreSQL — no Python math. The SLA source of truth is the priorities table (sla_first_response_min, sla_resolution_min) and the timestamps recorded in fact_ticket.